

Capitolul 4

Programare în limbaj de asamblare

CUPRINS

CAPITOLUL 4	1
4.1. SETUL DE REGISTRE LA PROCESOARELE INTEL	3
4.2. LIMBAJUL DE ASAMBLARE	5
4.2.1. Elementele limbajului	6
4.2.2. Definirea și utilizarea segmentelor	9
4.2.3. Lucrul cu întreruperi	11
4.2.4. Setul de instrucțiuni	12
4.3. EXEMPLE DE PROGRAME ÎN LIMBAJ DE ASAMBLARE	15
4.3.1. Hello, world!	16
4.3.2. Inversare	16
4.3.3. Conversie	17
4.3.4. Citește și afișează	18
4.3.5. Adunare	19
4.3.6. Scrie registru în hexazecimal	19
4.3.7. Scrie registru în zecimal	20
4.3.8. Numără 1	21
4.3.9. Lucru cu memoria ecran	22
4.3.10. Meniu	22
4.3.11. Maxim	23
4.3.12. Fibonacci	24

BIBLIOGRAFIE SPECIFICĂ

- [1] VLAD CĂPRARIU
Sistemul de operare DOS. Comenzi, Editura Microinformatica, Cluj-Napoca, 1993
- [2] VLAD CĂPRARIU
Sistemul de operare DOS. Ghidul programatorului, Editura Microinformatica, Cluj-Napoca, 1993
- [3] VLAD CĂPRARIU
Sistemul de operare DOS. Funcții sistem, Editura Microinformatica, Cluj-Napoca, 1991

4.1. Setul de registre la procesoarele Intel

Familia de procesoare 8086, 88, 186, 286 conține același set de registre de bază, instrucțiuni și moduri de adresare. În ceea ce privește noțiunile introductive despre programarea în asamblare, abordate în acest curs, diferențele care apar la procesoarele 386, 486 și respectiv Pentium nu sunt esențiale.

Procesorul 286 are 15 registre, grupate în patru categorii, conform tabelului următor:

Registre generale			
31	16	15	8 7 0
EAX		AX (AH)	(AL)
EBX		BX (BH)	(BL)
EDX		DX (DH)	(DL)
ECX		CX (CH)	(CL)
Registre pentru instrucțiuni de I/O, *, /			
Registru numărător			
Registre generale de index și de bază			
31	16	15	0
EBX		BX	
EBP		BP	
ESI		SI	
EDI		DI	
ESP		SP	
Registre de bază			
Registre de index			
Indicator de stivă			
Registre segment			
	CS		Code Segment
	DS		Data Segment
	ES		Extra Segment
	SS		Stack Segment
Registre de stare și control			
31	16	15	0
EIP		IP	
EF		F	
		MSW	
Instruction Pointer			
Flag			
Machine Status Word			

Registrele generale sunt:

a) **registrele de utilizare generală** se notează cu:

- AX – registrul acumulator general
- BX – registrul de bază
- CX – registrul numărător – folosit în operații de deplasare, rotire, bucle software, repetări hardware ale unor instrucțiuni;
- DX – registrul de date.

Acestea pot fi adresate în două moduri: direct pe 16 biți sau adresând separat byte-ul superior (primii 8 biți) și byte-ul inferior (următorii 8 biți). Corespunzător, cuvintele se vor numi:

15	7
8	0
AH	AL
BH	BL
CH	CL
DH	DL

La procesoarele Intel 80386, Intel 80486, fiecare dintre aceste registre este completat cu o extensie pe 2 octeți rezultând registrele EAX, EBA, ECX, EDX.

- b) **registrul indicator de stivă** de 16 biți. Acesta conține adresa vârfului stivei, SP (*stack pointer*).
- c) **registrul indicator de bază** de 16 biți. Acesta conține adresa de bază, BP (*base pointer*).
- d) **registrele de index**: SI (*source index*) și DI (*destination index*), de 16 biți fiecare. Aceștia participă la elaborarea adreselor, fiecare adresă rezultând prin însumarea diferitelor combinații între adresa de bază, un indice relativ la poziția curentă și o deplasare (numită *adresă de offset*).

De asemenea, la procesoarele moderne și acești regiștri au 2 octeți de extensie.

Registrele de segment.

Procesoarele Intel au patru registre de segment: câte unul pentru fiecare segment (de cod, de date, de stivă) și un registru de segment suplimentar (ExtraSegment). Fiecare dintre acești regiștri este pe 2 octeți (sau pe 4 octeți la procesoarele mai moderne), fără extra-registre.

Registrele de stare și control.

Registrul IP (*Instruction Pointer*) reține adresa de *offset* pentru următoarea instrucțiune din secvența ce va fi executată.

Cel mai important registru de stare este registrul indicatorilor de stare (registrul F, *Flags*). Cei 16 indicatori ai registrului de flag-uri (biți de stare, biți de control și indicatori speciali) au următoarea semnificație:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
X	NT	IO	PL	OF	DF	IF	TF	SF	ZF	X	AF	X	PF	X	CF

X - rezervați;

Biți de stare:

- CF (Carry Flag) – indicator de transport din MSB al rezultatului;
- PF (Parity Flag) – indicator de paritate;
- AF (Auxiliary carry Flag) – indicator de transport de la bitul 4 la bitul 5;
- ZF (Zero Flag) – indicator de rezultat nul;
- SF (Sign Flag) – indicator de semn negativ;
- OF (Overflow Flag) – indicator de depășire aritmetică;

Biți de control:

- TF (Trace Flag) – indicator pentru controlul execuției instrucțiunilor în regim pas cu pas în scopul depanării programelor;
- IF (Interrupt Flag) – indicator care controlează acceptarea semnalelor de întrerupere externă;
- DF (Direction Flag) – indicator de control pentru direcția de parcurgere a șirurilor de date;

Indicatori speciali:

- IOPL (Input – Output Privilege Level) – indicator special pe 2 biți care definește drepturile de utilizare a instrucțiunilor de intrare – ieșire;
- NT (Nested Task) – indicator pentru operațiile de comutare de task.

Registrul MSW (cuvântul de stare a mașinii) reține contextul curent de stare a procesorului astfel încât, în cazul apariției unei erori, starea procesorului să poată fi refăcută după depanare și programul procesorului să poată fi reluat dintr-un context corect de dinaintea apariției erorii.

4.2. Limbajul de asamblare

De ce utilizăm limbajul de asamblare? La această întrebare putem răspunde în mai multe moduri, în funcție de scopul pentru care inserăm în codul sursă o secvență de cod în asamblare.

Un prim răspuns s-ar putea referi la optimizarea utilizării resurselor: există componente ale sistemului de operare sau unele aplicații speciale care sunt considerate critice și trebuie să fie performante din punctul de vedere al

consumului de timp, memorie și resurse fizice. În astfel de situații se impune utilizarea cât mai eficientă a instrucțiunilor și structurii procesorului.

Din alt punct de vedere, un specialist în informatică trebuie să cunoască mecanismele fine ale procesorului pentru a le putea folosi în diferite aplicații.

În fine, cunoașterea instrucțiunilor unui limbaj de asamblare poate fi utilă pentru depanarea codului obiect al unui program oarecare.

Deși aceste argumente susțin necesitatea utilizării limbajului de asamblare, nu putem nega faptul că programarea în asamblare este mult mai dificilă decât cea într-un limbaj de nivel înalt. Aceasta în primul rând pentru că, în asamblare, pe lângă cunoștințele de limbaj, mai sunt necesare și cunoștințe legate de structura internă a calculatorului: regiștri, organizarea și adresarea memoriei, utilizarea porturilor de comunicare pentru operațiile de intrare / ieșire, ș.a.

Contrar tendințelor moderne de a putea rula o aplicație pe o diversitate cât mai mare de platforme, un dezavantaj al lucrului în asamblare este faptul că fiecare procesor are propriul limbaj de asamblare, deci programele nu pot fi rulate pe procesoare incompatibile (din familii diferite).

4.2.1. Elementele limbajului

În cele ce urmează ne vom referi la limbajul de asamblare al procesoarelor din familia Intel și cele compatibile cu acestea.

Limbajul de asamblare nu este *case-sensitive* (nu face distincție între literele mari și literele mici).

Alfabetul limbajului constă din literele alfabetului latin, cifrele arabe și caractere speciale.

Ca și în alte limbaje de programare, un identificator este o secvență de litere, cifre și caractere speciale, cu condiția ca primul caracter să fie o literă sau un caracter special din mulțimea: ?, _, @, \$.

Caracterul punct (.) înaintea unui identificator introduce o comandă pentru asamblor (o directivă de asamblare). De exemplu: .MODEL, .CODE ș.a.

Identificatorii pot fi împărțiți în două categorii: (1) identificatori standard și (2) identificatori definiți de utilizator.

Identificatorii standard pot denumi:

- instrucțiuni;
- nume de resurse (regiștri sau segmente);
- nume de operatori, de exemplu: MOD, OFFSET, SEG, TYPE, ș.a.;
- pseudoinstrucțiuni pentru asamblor, de exemplu: ASSUME, MODEL, ș.a..

Identificatorii definiți de utilizator pot fi

- nume de variabile;
- adrese de instrucțiuni;

- nume de operanzi;
- nume de proceduri;
- nume de segmente;
- macroinstrucțiuni.

În cele ce urmează vom lua în discuție pe rând principalele elemente al limbajului de asamblare: constante, propoziții, instrucțiuni, pseudoinstrucțiuni, operatori speciali.

Constante. Valoarea unei constante în asamblare depinde de baza în care este reprezentată valoarea respectivă. Aceasta se precizează prin următoarele litere standard: B – pentru binar, Q – pentru octal, D – pentru zecimal și H – pentru hexazecimal.

O constantă hexezecimală începe obligatoriu cu o cifră – eventual 0 (pentru a se deosebi de identificatori).

Propoziții. În asamblare, o propoziție este orice secvență de maxim 128 caractere acceptate de limbaj.

Instrucțiuni. Orice propoziție care este recunoscută de limbaj și pe care o traduce în cod mașină este o instrucțiune. Forma generală a unei instrucțiuni în asamblare (formatul de instrucțiune) este:

[<etichetă>] MNEMONICA [<operanzi>] [; comentariu]

unde

- ✓ <etichetă> este un câmp opțional care conține un nume simbolic de maxim 31 de caractere. Fiecare etichetă are asociată o valoare numerică ce reprezintă adresa relativă în cadrul segmentului din care face parte;
- ✓ MNEMONICA este numele instrucțiunii;
- ✓ <operanzii> depind de instrucțiunea propriu-zisă; dacă instrucțiunea cere doi operanzi atunci aceștia se vor numi DESTINAȚIE și SURSĂ și apar în codul simbolic al instrucțiunii în această ordine.

Declararea datelor. Operatorul DUP. Pentru declararea datelor în asamblare se folosește o pseudoinstrucțiune care asigură atât alocarea de memorie pentru date și specificarea tipului datelor, cât și inițializarea datelor.

Formatul de declarare a datelor este:

[<nume_variabilă> tip [<listă_expresii>] [; comentariu]

sau

[<nume_variabilă> tip [<număr>] DUP ([<listă_expresii>]) [; comentariu]

unde

- ✓ <tip> reprezintă instrucțiunea folosită pentru declararea de date; acesta poate fi un nume de structură sau poate să ia valorile:
 - DB – pentru date de tip BYTE (reprezentate pe 1 octet);
 - DW – pentru date de tip WORD (2 octeți);

- DD – pentru date de tip DOUBLE (4 octeți);
 - DQ – pentru date de tip QUADRUPLE (8 octeți);
 - DF – pentru date de tip FLOAT (valori reale pe 6 octeți);
 - DP – pentru date de tip POINTER;
 - DT – pentru date pe 10 octeți (ten-bytes);
- ✓ <listă_expresii> este lista valorilor inițiale ale datelor declarate prin această instrucțiune; pe poziția variabilelor care nu se inițializează se va trece caracterul “?”;
- ✓ <număr> **DUP** (<listă_expresii>) definește o structură formată din <număr> elemente; operatorul DUP (engl. *duplicate*) permite multiplicarea definirii datelor.

Operatorul INDEX. Simbolul operatorului este perechea de paranteze drepte “[]”. Operatorul reprezintă intern o adunare pentru că este folosit pentru modurile de adresare indirectă, indexată, cu registru de bază a operanzilor.

Exemplu. A[bx][di] referă adresa A[bx+di] și adresa A + (bx) + (di) și adresa [A + bx + di], unde prin (zz) am referit conținutul registrului zz, iar prin (yy) am referit conținutul de la adresa yy.

Operatorii relaționali recunoscuți de limbajul de asamblare sunt:

- ✓ EQ (equal) – pentru egalitate;
- ✓ NE (not equal) – pentru diferit;
- ✓ GT (greater than) – pentru mai mare;
- ✓ GE (greater than or equal to) – pentru mai mare sau egal;
- ✓ LT (less than) – pentru mai mic;
- ✓ LE (less than or equal to) – pentru mai mic sau egal.

Operatori de tip și de conversie.

Operatorii HIGH și LOW selectează octetul cel mai semnificativ, respectiv cel mai puțin semnificativ al unei valori reprezentată într-un registru pe 2 octeți.

Operatorul OFFSET returnează deplasamentul în cadrul segmentului curent pentru expresia pe care se aplică. Această expresie poate fi o variabilă, un operand de memorie, o etichetă.

Operatorul TYPE returnează un întreg ce reprezintă tipul expresiei pe care se aplică operatorul. Astfel, pentru valori de tip BYTE se returnează 1, pentru WORD – 2, pentru DOUBLE – 4, ș.a.m.d; pentru o structură, operatorul returnează numărul de octeți din structura respectivă.

Operatorul LENGTH returnează o valoare întreagă ce reprezintă numărul de elemente ale unei variabile declarate cu DUP și 1 pentru variabile nedeclarate cu DUP.

Directiva EQU. Forma generală de utilizare este:

<nume> EQU <expresie>

Directiva permite asignarea la asamblare a valorii dată de <expresie> la simbolul dat de <nume>.

Pentru valori numerice utilizarea acestei directive este echivalentă cu atribuirea (operatorul "=").

Simbolul CONTOR PROGRAM, „\$”. Simbolul „\$” reprezintă adresa curentă a contorului de program, adică adresa relativă în cadrul segmentului curent a instrucțiunii sau a datelor de asamblat.

La începutul fiecărui segment, contorul de locații (\$) este inițializat cu 0 (zero) și este actualizat automat de asamblor pe măsură ce se assemblează programul.

Contorul de program (\$) poate fi inițializat explicit folosind directiva ORG.

4.2.2. Definirea și utilizarea segmentelor

Pentru procesoarele Intel memoria este organizată ca seturi de segmente de lungime variabilă. Fiecare segment este o secvență contiguă, liniară, de până la 64 KO (2^{16} biți). Adresa fizică de început a unui segment este multiplu de 16. Fiecare segment este alcătuit din locații succesive de memorie și este o unitate independentă și adresabilă separat. Fiecărui segment *i* se asociază o adresă de bază în spațiul de memorie – care este adresa de start a segmentului respectiv.

Segmentele pot fi adiacente, disjuncte sau suprapuse parțial sau total. O locație de memorie poate fi conținută în mai multe segmente. Unei locații de memorie *i* se asociază o adresă fizică – care identifică în mod unic locația respectivă – și o adresă logică – cu care lucrează programatorul și care nu este unică ci depinde de logica și contextul programului care o folosește. Adresa logică se compune dintr-un **selector** (dat de adresa de bază a segmentului) și un **deplasament** (dat de *distanța* față de începutul segmentului). Practic, aceeași adresă fizică poate fi referită de mai multe adrese logice obținute prin modificarea în mod corespunzător a deplasamentului față de adresele de început a diferite segmente. Referirea la o variabilă sau etichetă din alt segment se poate face atât prin adresă fizică, cât și prin adresă logică (adresa de start : deplasament).

În cele ce urmează prin **segment** vom înțelege o colecție de instrucțiuni sau de date ale căror adrese sunt relative față de adresa de început a segmentului. Această adresă de start este reținută de registrul segment corespunzător: CS (pentru cod sau program), DS (pentru date), SS (pentru stivă). Programatorul poate folosi direct acești regiștri sau poate defini segmente logice, câte unul pentru fiecare dintre segmentele fizice standard (cod, date, stivă).

În funcție de maniera de lucru aleasă de programator, acesta va folosi una dintre variantele de specificare a segmentelor: (1) prin definire simplificată sau (2) prin definire completă.

(1) Directive simplificate de segmentare: .MODEL, .STACK, .DATA, .CODE

Dacă s-a optat pentru specificarea simplificată a segmentelor atunci prima directivă din programul scris în limbaj de asamblare va fi

`.MODEL <tip>`

care specifică dimensiunile și modul dispunere a segmentelor în memoria RAM. Automat, această instrucțiune se assemblează ca o instrucțiune ASSUME implicită (vezi în continuare). La încărcarea în memoria RAM a unui program pentru execuție, sistemul de operare inițializează registrul CS cu prima adresă de segment disponibilă și registrul IP cu adresa relativă (în cadrul segmentului) a primei instrucțiuni de executat.

Parametrul <tip> poate lua valorile: tiny, small, medium, compact, large, huge.

Directiva

`.STACK <dim>`

determină asamblorul să inițializeze registrul SP cu valoarea *dim* care reprezintă dimensiunea segmentului de stivă. Implicit, această dimensiune este 1KO.

Directiva

`.DATA`

precede zona de program pentru declarații de date.

Observație. Programatorul trebuie să inițializeze EXPLICIT la începutul codului registrul DS cu adresa de început a segmentului de date dată de @data. Aceasta se realizează prin două instrucțiuni MOV astfel:

`MOV AX, @data`

`MOV DS, AX`

Directiva

`.CODE <nume>`

precede programul propriu-zis. Parametrul *nume* este ignorat pentru modelele tiny, small, compact. Implicit, *nume* este @CODE.

(2) Definirea completă a segmentelor. Instrucțiunile SEGMENT și ASSUME

Instrucțiunea SEGMENT pentru definirea segmentelor utilizator are sintaxa:

`nume_segment SEGMENT <caracteristici>`

corpul segmentului: instrucțiuni sau date

`nume_segment ENDS`

La execuție, entității *nume_segment* i se asociază o adresă de segment pe 2 octeți corespunzătoare poziției în memorie a segmentului definit.

Ulterior, programatorul trebuie să inițializeze un registru segment (DS, ES sau SS) cu segmentul declarat. De exemplu, pentru legarea segmentului de segmentul de date se vor folosi instrucțiunile:

```
MOV AX, nume_segment
```

```
MOV DS, AX
```

Dacă segmentul de date a fost definit astfel atunci în codul respectiv programatorul trebuie să aibă grijă ca următoarele referiri la memorie (care implică segmentul DS) sunt referiri la date din acest segment.

Un segment utilizator definit cu `SEGMENT` trebuie asociat unui registru segment standard. Aceasta se realizează cu instrucțiunea `ASSUME`:

```
ASSUME <registru_segment> : <nume_segment>
```

unde <registru_segment> poate lua valorile CS, DS, ES, SS, iar

<nume_segment> referă unul dintre segmentele declarate anterior cu `SEGMENT`.

Implicit, registrul segmentului definit de utilizator va fi adresat de registrul segment standard cu care este asociat prin `ASSUME`.

4.2.3. Lucrul cu întreruperi

În general, o întrerupere determină transferul execuției programului la o nouă adresă de program (adresa de început a rutinei de tratare a întreruperii respective). Implicit, atât vechea adresă a instrucțiunii curente din programul în execuție (dată de CS:IP) cât și starea procesorului (registrul de stare și control) sunt salvate pe stivă pentru refacerea contextului.

Întreruperile se pot grupa în trei clase: (1) întreruperi inițiate de hardware – ca răspuns la un semnal extern, (2) întreruperi determinate prin program – generate de instrucțiunea `INT` și (3) excepții în execuția unor instrucțiuni. În cele ce urmează ne vor interesa întreruperile din grupa a doua.

Întreruperile generate de instrucțiunea `INT` sunt utilizate curent pentru testarea procedurilor de tratare a întreruperilor sau pentru a apela servicii DOS (servicii ale sistemului de operare).

Instrucțiunea `INT` cu sintaxa:

```
INT n
```

generează prin software un apel de procedură de întrerupere.

Parametrul `n` reprezintă indexul din tabela IDT (Interrupt Descriptor Table) a rutinei de întrerupere ce va fi apelată. Valoarea parametrului `n` se reprezintă pe 1 octet, deci se pot apela 256 întreruperi (`n` poate lua valori naturale de la 0 la 2^8-1). Dintre acestea, primele 32 sunt rezervate de producătorul Intel, celelalte fiind la dispoziția programatorului. Valoarea se dă în hexazecimal (deci pe două poziții hexa, $255_{(10)} = FF_{(16)} = Fx16 + F$) și de aceea trebuie urmată de caracterul `h` (sau `H`).

Exemple.

`INT 10h` este instrucțiunea de apelare a serviciilor video oferite de BIOS; la apelul acestei întreruperi ea trebuie să găsească în octetul AH al registrului AX codul serviciului solicitat. De exemplu, codul 00h determină selectarea modului video;

INT 21h este instrucțiunea de apelare a funcțiilor DOS; pentru fiecare apel de funcție DOS se procedează în mai mulți pași:

1. se încarcă numărul funcției solicitate în octetul AH;
2. dacă este necesar se încarcă numărul subfuncției în octetul AL;
3. dacă este necesar se încarcă celelalte date în regiștri specifici de lucru pentru funcția DOS respectivă;
4. se generează întreruperea 21h.

Funcții DOS uzuale:

- 01h – citirea cu ecou a unui caracter de la tastatură; funcția returnează în octetul AL caracterul citit;
- 08h – citirea fără ecou (caracterul tastat nu este afișat pe ecran);
- 02h – afișarea unui caracter; codul caracterului trebuie să fie în octetul DL;
- 09h – afișarea unui șir de date; secvența de date trebuie să se termine cu caracterul '\$'; registrul DX trebuie să conțină deplasamentul adresei de început a șirului față de adresa segment din DS;
- 05h – tipărirea unui caracter;
- 4Ch – terminarea procesului (EXIT); apelarea acestei funcții determină revenirea în DOS la încheierea execuției programului respectiv în asamblare;
- 3Fh – citirea dintr-un fișier sau de la un dispozitiv dat prin identificator logic;
- 40h – scrierea într-un fișier sau la un dispozitiv dat prin identificator logic.

4.2.4. Setul de instrucțiuni

Din punctul de vedere al funcției lor, instrucțiunile limbajului de asamblare se împart în trei grupe:

- A. setul instrucțiunilor de bază;
- B. setul extins de instrucțiuni;
- C. setul instrucțiunilor de controlul sistemului.

Pentru fiecare instrucțiune vom da mnemonica, sintaxa, semnificația operanzilor și câteva reguli importante pentru utilizarea instrucțiunii respective.

A. Instrucțiuni de bază

1. Instrucțiuni de transfer

- a) Instrucțiuni de transfer cu scop general

MOV <destinație>, <sursă>

- transferă un octet (un cuvânt) de la <sursă > la <destinație>;
- registrul CS nu poate fi destinație;
- nu se pot transfera date direct între două registre segment;
- nu se pot transfera date direct între două locații de memorie;

CWB – instrucțiune de conversie

XCHG <destinație> <sursă> – instrucțiune de interschimbare

PUSH <sursă> – instrucțiune de încărcare pe stivă

- operandul <sursă> poate fi: un registru general, o locație de memorie de 2 octeți, un registru segment,

POP <destinație> – instrucțiune de descărcare de pe stivă

PUSHA – instrucțiune care salvează pe stivă regiștrii: AX, CX, DX, BX, SP, BP, SI, DI, în această ordine;

POPA – încarcă registrele de mai sus cu date de pe stivă (evident, în ordine inversă);

b) Instrucțiuni de transfer specifice registrului acumulator

Aceste instrucțiuni transferă date între AX, AL, EAX, și portul de intrare / ieșire din spațiul I/O specificat ca operand.

c) Instrucțiuni de transfer al adreselor obiect

LEA <destinație_registru>, <sursă_memorie>

- (engl., *Load Effective Address*) – încarcă în registru destinație adresa de memorie dată ca sursă;
- Registrul destinație nu poate fi un registru segment;

d) Instrucțiuni de transfer pentru registru de stare

2. Instrucțiuni aritmetice

Numerele binare cu semn sunt reprezentate intern în cod complementar (în complement față de 2) și pot avea valorile întregi între -128 și 127 (adică între -2^7 și 2^7-1 , un bit fiind rezervat pentru semn) – pentru cele pe un octet – și între -32768 și 32767 (adică între -2^{15} și $2^{15}-1$) – pentru cele pe 2 octeți.

a) Adunare / scădere

ADD <destinație>, <sursă>

- destinație $\leftarrow$ destinație + sursă
- implicit, instrucțiunea modifică următoarele flaguri (biți din registru de stare): OF, SF, ZF, AF, PF, CF;

ADC <destinație>, <sursă>

- (engl., *ADdition with Carry flag*) – adună și valoarea curentă a bitului de transport;

INC <destinație>

- destinație $\leftarrow$ destinație + 1

DEC <destinație>

- destinație $\leftarrow$ destinație - 1

SUB <destinație>, <sursă>

- destinație $\leftarrow$ destinație - sursă - (CF)

b) Înmulțire / împărțire

MUL <sursă>

- înmulțește operandul <sursă> cu acumulatorul (AX, AL, EAX);

DIV <sursă>

- execută împărțire întreagă a acumulatorului prin operandul <sursă>

3. Instrucțiuni de prelucrare la nivel de bit

a) Instrucțiuni logice: NOT, AND, OR, XOR

b) Instrucțiuni de deplasare: SHR, SHL

c) Instrucțiuni de rotire: ROL, ROR

4. Instrucțiuni de operare pe șiruri de date

REP (engl., *REPeat*) – repetă cât timp nu este sfârșitul șirului;

MOVS (engl., *MOVE data from String to string*)

C. Instrucțiuni de control

1. Instrucțiuni pentru controlul programului

a) Instrucțiuni de transfer necondiționat

CALL <nume_procedură>

- apelează procedura cu numele dat ca operand;
- salvează pe stivă adresa de revenire

RET [<adresă_de_revenire>]

- (engl., *RETurn*) – revenire în programul apelant;

JMP <destinație>

- (engl., *JuMP*) – salt la adresa destinație;
- nu se salvează pe stivă nici o informație;

b) Instrucțiuni de transfer condiționat

Jcond <destinație>

- aici cond reprezintă sufixul numelui instrucțiunii, sufix care depinde de condiția de salt;

- condiția se pune în raport cu starea indicatorilor la momentul execuției instrucțiunii;
- c) Instrucțiuni de ciclare
- LOOP <etichetă>
- instrucțiunea se execută implicit cu conținutul registrului CX; bucla se repetă până când conținutul lui CX este 0;
- d) Instrucțiuni de întrerupere
- INT <tip_întrerupere>
- instrucțiune de apel de întrerupere
2. Instrucțiuni pentru controlul procesorului
- a) Instrucțiuni pentru poziționarea indicatorilor
- CLC (engl., *CLear Carry flag*)
- STC (engl., *SeT Carry flag*)
- CLD (engl., *CLear Direction flag*)
- STD (engl., *SeT Direction flag*)
- b) Instrucțiuni pentru sincronizare cu evenimente externe: HLT, WAIT, ESC
3. Instrucțiuni pentru limbaje structurate pe blocuri: ENTER, LEAVE
4. Proceduri
5. Instrucțiuni pentru organizarea modulară a programelor
6. Macroinstrucțiuni

4.3. Exemple de programe în limbaj de asamblare

Etapele de lucru în asamblare. Pentru a rula un program scris în limbaj de asamblare se va proceda astfel:

1. se scrie programul folosind un editor de texte oarecare;
2. se salvează programul cu extensia „asm”;
3. se caută fișierele tasm și tlink (asamblorul și linkeditorul) și se setează calea acestor fișiere prin comanda DOS path <cale>;
4. se execută asamblarea fișierului sursă (salvat la 2.) cu comanda DOS tasm <NumeFișier.asm>; dacă în etapa de asamblare se detectează erori atunci acestea sunt listate pe ecran (fiecare eroare precedată de numărul liniei din codul sursă pe care a apărut); dacă asamblarea s-a încheiat cu succes atunci se generează automat fișierul obiect (cu același nume cu fișierul sursă și cu extensia .obj);
5. se execută linkeditarea fișierului obiect generat la 4. folosind comanda tlink <NumeFișier.obj>; în urma linkeditării rezultă fișierul executabil;
6. se rulează fișierul executabil anterior (execuția propriu-zisă).

Observație. După ce aceste etape sunt parcurse pentru câteva programe distincte, se poate crea un fișier *batch* (fișier.bat) pentru micșorarea timpului de lucru.

Pentru depanarea programului, respectiv execuția pas cu pas se va folosi comanda td <NumeFișier.exe>.

4.3.1. Hello, world!

Programul *Hello, world!* afișează pe ecran mesajul *Hello, world!*.

```
.model small
.stack 10
.data
    mesaj db 'Hello, world!$'
.code
start:
    ; inițializarea registrului DS
    mov ax, @data
    mov ds, ax
    ; mută în DX adresa de început a datei mesaj
    mov dx, offset mesaj
    ; scrie pe ecran conținutul registrului DX, până la caracterul $
    mov ah, 9
    int 21h
    ; revenirea în DOS
    mov ax, 4c00h
    int 21h
ens start
```

4.3.2. Inversare

Programul inversează între ei octeții dintr-un șir de cuvinte de memorie.

```
.model small

.data
mesaj dw 'Ex', 'em', 'pl', 'u ', 'pr', 'og', 'ra', 'm ', '2$'
lung_mesaj equ ($-mesaj)/2
linie_noua db 0dh, 0ah, '$'

.code
start:
    mov ax, @data
    mov ds, ax
    lea dx, mesaj
    mov ah, 9
    int 21h
    lea dx, linie_noua
    mov ah, 9
    int 21h
    lea si, mesaj
    mov cx, lung_mesaj

iar:  mov ax, [si]
      xchg al, ah
      mov [si], ax
```



```

    add    si, type mesaj
    loop   iar

    lea    dx, mesaj
    mov    ah, 9
    int    21h
    mov    ax, 4c00h
    int    21h
end start

```

4.3.3. Conversie

Programul convertește un număr din baza 10 în baza 8.

```

.model small
.stack 100h

.data
    sir DB 80 DUP(0)

.code
mov  bx, 0
mov  dx, offset sir
mov  cx, 80
mov  ah, 3fh
int  21h

mov  bx, offset sir
mov  cx, ax      ;cx contine lungimea sirului
mov  ax, 0       ;ax devine 0

iteratie:        ;in cx ramane numarul de cifre al numarului in baza 16
    sub  [bx], '0'
    mov  dx, 10
    mul  dx      ;inmulteste continutul lui ax cu dx si memoreaza in dx
    mov  dh, 0
    mov  dl, [bx]
    add  ax, dx;la ax adauga continutul lui dx (respectiv dl)
    add  bx, 1 ;bx contine nr cifrelor care au fost prelucrate
    sub  cx, 1 ;cx se micsoreaza cu o unitate
    cmp  cx, 2 ;compara cx cu 2 deoarece la sir se mai adauga automat 2
                caractere implicite
    jne  iteratie

    mov  cx, 0    ;initializeaza registrul cx
    mov  bx, 8    ;baza de conversie
    mov  dx, 0    ;initializeaza registrul dx

ciclul:
    div  bx      ;imparte continutul lui ax la bx si retine restul in dx si catul
                in ax
    push dx      ;pune restul in stiva
    mov  dx, 0 ;initializeaza dx
    add  cx, 1 ;incrementeaza cx pentru a retine nr de elem din stiva
    cmp  ax, 0 ;daca in ax catul impartirii este 0 atunci iese din ciclu
    jne  ciclul

    mov  bx, offset sir ;in bx muta adresa inceputului de sir
    mov  dx, cx         ;dx va contine nr de resturi obtinute la impartire

```

```

ciclu2:
    pop     ax          ;se extrage din stiva primul rest in ax
    mov     [bx], ax    ;muta la adresa din bx continutul lui ax
    add     [bx], '0'    ;transforma continutul de la adresa din bx in caracter
    add     bx, 1        ;incrementeaza bx
    sub     cx, 1        ;decrementeaza cx
    cmp     cx, 0        ;compara cx cu 0
    jne     ciclu2

    mov     cx, dx        ;cx devine 0
    mov     bx, 1        ;deschide fisierul spre scriere
    mov     dx, offset sir ;dx contine adresa inceputului de sir
    mov     ah, 40h       ;ah contine codul functiei de scriere
    int     21h

    mov     ah, 4ch       ;terminare
    int     21h
end

```

4.3.4. Citește și afișează

Programul citește de la tastatură o tastă și scrie codul ASCII al acelei taste.

```

seg_date segment
    tab_conv      db    '0123456789ABCDEF'
    mesaj         db    '- are codul ASCII: '
    tasta         db    2 dup (?), 0dh, 0ah, '$'
seg_date ends

seg_cod segment
    assume cs: seg_cod, ds: seg_date
    start:
        mov     ax, seg_date
        mov     ds, ax
        mov     ah, 1        ; citeste cu ecou o tasta
        int     21h
        mov     ah, al       ; salveaza codul tastei
        lea     bx, tab_conv; initializeaza bx pentru xlat
        and     al, 0fh      ; se retine numai al doilea grup de 4 biti

        xlat     tab_conv     ; codul ASCII al acestui grup de 4 biti este

        mov     tasta+1, al   ; a doua cifra a codului
        mov     al, ah        ; codul initial al tastei
        mov     cl, 4         ; numara deplasarile la dreapta
        shr     al, cl        ; deplasare logica la dreapta cu 4 biti

        xlat     tab_conv     ; conversia primului grup de 4 biti

        mov     tasta, al     ; codul ASCII al primului grup de 4 biti

        lea     dx, mesaj
        mov     ah, 9         ; tipareste codul tastei
        int     21h
        mov     ax, 4c00h     ; revenire in DOS
        int     21h

```

```
seg_cod ends
end start
```

4.3.5. Adunare

Programul adună două numere reprezentate pe mai mulți octeți și depune rezultatul peste primul număr.

```
add_data segment
    numar_1    dw    0a8adh, 7fe2h, 0a876h, 0
    lung_nrl   equ    ($ - numar_1)/2
    numar_2    dw    0cdefh, 52deh, 378ah, 0
add_data ends

multibyte_add segment
assume cs:multibyte_add, ds: add_data
start:
    mov     ax, add_data    ; init reg ds
    mov     ds, ax         ; cu adresa de segment
    mov     cx, lung_nrl    ; contor = lungime nrl
    mov     si, 0           ; init index de elem
    cld                     ; (cf) = 0 transportul initial
    bucla:
        mov     ax, numar_2[si]
        adc     numar_1[si], ax
        inc     si          ; actualiz index de elem
        inc     si
    loop    bucla
    mov     ax, 4c00h        ; revenire in DOS
    int     21h
multibyte_add ends
end start
```

4.3.6. Scrie registru în hexazecimal

Programul scrie valoarea hexazecimală conținută de registrul acumulator AX.

```
.model small

.data
tab_conv db    '0123456789ABCDEF'
nr        dw    ?

.code
start:
    mov     ax, @data
    mov     ds, ax

    mov     bx, 0fa3dh
    mov     nr, bx

    mov     ax, [nr]
    and     ah, 0f0h
    lea     bx, tab_conv
    mov     al, ah
```

```

mov     cl, 4
shr     al, cl
xlat    tab_conv
mov     dl, al
mov     ah, 2
int     21h

mov     ax, [nr]
and     ah, 0fh
lea     bx, tab_conv
mov     al, ah
xlat    tab_conv
mov     dl, al
mov     ah, 2
int     21h

mov     ax, [nr]
and     al, 0f0h
lea     bx, tab_conv
mov     cl, 4
shr     al, cl
xlat    tab_conv
mov     dl, al
mov     ah, 2
int     21h

mov     ax, [nr]
and     al, 0fh
lea     bx, tab_conv
xlat    tab_conv
mov     dl, al
mov     ah, 2
int     21h

mov     ah, 4ch
int     21h
end start

```

4.3.7. Scrie registru în zecimal

Programul scrie conținutul registrului AX convertit în zecimal.

```

.model small
.data
nr      dw 0a2d3h
mesaj   db '(16)A2D3 = (10)$'

.code
start:
    mov     ax, @data
    mov     ds, ax

    lea     dx, mesaj
    mov     ah, 9
    int     21h

    mov     ax, [nr]

    mov     bx, 100 ; impartitor

```

```

mov     dx, 0      ; extensie semn, deimpartit pozitiv
div     bx         ; dx = ultimile 2 cifre: zeci si unitati
mov     cx, dx     ; ax = primele 3 cifre, cx = ultimile 2 cifre
mov     dx, 0      ; extensie semn, deimpartit pozitiv
div     bx         ; dx = cifre: mii, sute; ax = cifra zeci de mii
push    dx         ; se salveaza dx deoarece va fi modificat
add     al, 30h    ; se tipareste cifra zecilor de mii
mov     dl, al
mov     ah, 2
int     21h

pop     ax         ; se iau din stiva valorile pentru mii si sute
aam                     ; conversie la format zecimal neimpachetat
add     ax, 3030h  ; conversie la cod ASCII
mov     dx, ax     ; se salveaza cele doua cifre
xchg    dh, dl     ; se tipareste cifra miilor
mov     ah, 2
int     21h

xchg    dh, dl     ; se tipareste cifra sutelor
mov     ah, 2
int     21h

mov     ax, cx     ; ultimile doua cifre: zeci, unitati
aam                     ; conversie la format zecimal neimpachetat
add     ax, 3030h
mov     dx, ax     ; se salveaza cele doua cifre
xchg    dh, dl     ; se tipareste cifra zecilor
mov     ah, 2
int     21h

xchg    dh, dl     ; se tipareste cifra unitatilor
mov     ah, 2
int     21h

mov     ah, 4ch
int     21h
end start

```

4.3.8. Numără 1

Programul numără biții 1 din reprezentarea valorii unei variabile.

```

.model small
.stack 100h
.data
    lung_var equ 1
    var dw lung_var dup (0acfh)
    nr_1 db ?

.code
    assume cs: @code, ds: @data
start:
    mov     ax, @data
    mov     ds, ax

    mov     si, 0      ; index curent pentru cuvintele din variabila
    mov     dl, lung_var ; contor nr cuvinte
    mov     bl, 0      ; contor nr de 1 gasiti

```

```

    bucla2:
        mov     cx, 16                ; contorul de biti pentru un cuvânt
        mov     ax, var[si] ; se citește cuvântul curent
        bucla1:
            rcl   ax, 1                ; o rotație pentru a deplasa un bit
            adc   bl, 0                ; se contorizează nr de 1 dintr-un cuvânt
        loop    bucla1
        add     si, type var          ; se actualizează indexul
        dec     dl                    ; se testează dacă mai sunt cuvinte
        jnz     bucla2                ; dacă da, se reia citirea cuvintelor
        mov     nr_1, bl              ; dacă nu, se depune rezultatul

        mov     ax, 4c00h
        int     21h
    end start

```

4.3.9. Lucru cu memoria ecran

Programul copiază informația dintr-un buffer de memorie în memoria ecran.

```

.model small
.data
mem_ecran dd 0B8000000h ; adresa memoriei ecran
dim equ 2
buffer dw dim dup (2a2ah); se va afișa acest caracter de dim ori
ptr_buf dd buffer

.code
start:
    mov     ax, @data
    mov     ds, ax

    mov     ah, 0                ; selectează mod de lucru alfanumeric
    mov     al, 0                ; 40*25, alb/negru; pentru 80*25 se da al = 2
    int     10h

    les     di, mem_ecran
    lds     si, ptr_buf
    mov     cx, dim
    rep     movsw                ; transferă buffer în memoria ecran

    mov     ah, 4ch
    int     21h
.stack 10h
end start

```

4.3.10. Meniu

Programul execută una din mai multe secvențe de program, în funcție de răspunsul utilizatorului.

```

.model small
.data
executie db 'Executa secventa: 1, 2, 3 sau 4=stop: $'
mesaj1 db 0dh, 0ah, 'Se executa secventa 1.', 0dh, 0ah, '$'

```

```

    mesaj2      db    0dh, 0ah, 'Se executa secventa 2.', 0dh, 0ah, '$'
    mesaj3      db    0dh, 0ah, 'Se executa secventa 3.', 0dh, 0ah, '$'
    tab_secv    label word
                    dw    secv1
                    dw    secv2
                    dw    secv3
                    dw    gata

.code
start: mov     ax, @data
      mov     ds, ax

iar:   lea     dx, executie ; solicita utilizatorului nr secv dorite
      mov     ah, 9
      int     21h
      mov     ah, 1
      int     21h

      sub     al, 31h      ; transf interv 1..4 in 0..3
      cmp     al, 0       ; verific daca val citita este in interv 0..3
jc iar      ; daca nu este in interv atunci
      cmp     al, 4       ; se resolicita o val de la utiliz
jnc iar

      cbw                     ; extensie pe 16 biti
      mov     bx, ax
      shl     bx, 1        ; inmul cu 2 pt a se obtine adresa relativa
                        ; in tabela cu adresele secventelor de exec
      jmp word ptr tab_secv[bx]

secv1: lea     dx, mesaj1
      mov     ah, 9
      int     21h
      jmp short iar
secv2: lea     dx, mesaj2
      mov     ah, 9
      int     21h
      jmp short iar
secv3: lea     dx, mesaj3
      mov     ah, 9
      int     21h
      jmp short iar
gata:  mov     ah, 4ch
      int     21h
.stack 20h
end start

```

4.3.11. Maxim

Programul determină valoarea maximă dintr-un șir de valori.

```

.model small
.stack 10h
.data
    sir         db    10, 20, -30, 100, -100, 200
    lung        dw    $-sir
    max         db    ?
    MesSirVid db    'Sirul nu are valori incarcate.$'
.code
    mov     ax, @data
    mov     ds, ax

```

```

        mov     cx, lung           ; contor = nr val din sir
        jcxz    sir_vid           ; daca sir vid atunci salt la etich
sir_vid
        lea     si, sir
        cld                               ; Clear Direction Flag
        mov     bl, [si]           ; init max cu primul elem din sir
        inc     si
        dec     cx
        jcxz    gata
iar:     lodsb                      ; citeste elem crt din sir
        cmp     bl, al
        jge     urm
        mov     bl, al
urm:     loop   iar
gata:    mov     max, bl
iesire:  mov     ah, 4ch
        int     21h
sir_vid: lea     dx, MesSirVid
        mov     ah, 9
        int     21h
        jmp     iesire
end

```

4.3.12. Fibonacci

Programul determină primele N numere Fibonacci.

```

.model small
.data
n      dw      17
fibo   dw      ?

.code
start:
        mov     ax, @data;
        mov     ds, ax

        mov     ax, 1
        mov     bx, 0
        mov     cx, N
        mov     di, bx
urm:    mov     si, ax
        add     ax, bx
        mov     bx, si
        mov     fibo[di], ax
        add     di, type fibo
loop   urm

        mov     ah, 4ch
        int     21h
end start

```